

MALAYALAM NLP: DATASET, TRIAL SCRIPT AND PROTOTYPE:- P.V.MRIYA MUNESH 25MSDFR039

1. DATASET CREATED:-

ID	Text	Label
1	ഈ ഫോൺ വളരെ നല്ലതാണ്. ഈ സിനിമ എനിക്ക് വളരെ	positive
2	ഇഷ്ടപ്പെട്ടു.	positive
3	ഇത് വളരെ മോശമാണ്.	negative
4	സേവനം വളരെ ദയനീയമാണ്.	negative
5	ശരാശരി അനുഭവം.	neutral
6	ഉൽപ്പന്നം ഗുണമേന്മയുള്ളതാണ്.	positive
7	ഡെലിവറി വളരെ വൈകി.	negative
8	ഇത് ഒക്കെ മിതമായതാണ്.	neutral
9	വളരെ സന്തോഷം ഉണ്ട്.	positive
10	ഇത് പണം പാഴായി.	negative

Similarly dataset created until 2100 rows.

A	B	C	D	E
ID	Text	Label		
1	ഈ ഫേഡ്	positive		
2	ഈ സിനിമ	positive		
3	ഇത് വളരെ	negative		
4	സേവനം	negative		
5	ശരാശരി	neutral		
6	ഉൽപ്പന്നം	positive		
7	ഡെലിവറി	negative		
8	ഇത് ഒക്കെ	neutral		
9	വളരെ നല്ല	positive		
10	ഇത് പല	negative		
11	സിനിമ	neutral		
12	ഈ ആദ്യ	positive		
13	മോശം	negative		
14	ഒന്നും പ്രശ്നം	neutral		
15	വളരെ നല്ല	positive		
16	ഇത് എത്ര	negative		
17	ഉത്തമം	positive		
18	ഡിനൈറ്റ്	neutral		
19	പൂർണ്ണ	negative		
20	ഈ പുസ്തകം	positive		

Dataset created in excel sheet with sentiment value in csv file format, named as "Malayalam_reviews.csv"

2. TRIAL MODEL:-

```
# train_malayalam_bert.py
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from datasets import Dataset, DatasetDict
```

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
TrainingArguments, Trainer
```

```
# -----
```

```

# 1. Load Malayalam dataset
# -----

df = pd.read_csv("Malayalam_reviews.csv") # columns: id, text, label
df["Text"] = df["Text"].astype(str).str.strip()

# Ensure 'Label' column is string type to handle potential mixed types (e.g., NaN)
df["Label"] = df["Label"].astype(str)

# Filter out rows where 'Label' is 'nan' (string representation of NaN)
df = df[df["Label"] != "nan"].reset_index(drop=True)

# Encode labels
label_list = sorted(df["Label"].unique())
label2id = {lab: i for i, lab in enumerate(label_list)}
id2label = {i: lab for i, lab in enumerate(label_list)}
df["label_id"] = df["Label"].map(label2id)

print("Labels found:", label_list)
print("label2id:", label2id)
print("id2label:", id2label)

# -----

# 2. Split data → train / validation / test
# -----

train_df, temp_df = train_test_split(
    df, test_size=0.2, random_state=42, stratify=df["label_id"]
)

```

```
)
```

```
valid_df, test_df = train_test_split(  
temp_df, test_size=0.5, random_state=42, stratify=temp_df["label_id"]  
)
```

```
def to_hf_dataset(df_part):  
    return Dataset.from_pandas(df_part[["Text", "label_id"]])
```

```
dataset = DatasetDict({  
    "train": to_hf_dataset(train_df),  
    "validation": to_hf_dataset(valid_df),  
    "test": to_hf_dataset(test_df),  
})
```

```
# -----
```

```
# 3. Tokenizer
```

```
# -----
```

```
# mBERT supports Malayalam as well
```

```
model_name = "bert-base-multilingual-cased"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
def tokenize(batch):
```

```
    return tokenizer(  
        batch["Text"],
```

```
        truncation=True,
```

```
        padding="max_length",
```

```
padding="max_length",  
max_length=64,  
)
```

```
dataset = dataset.map(tokenize, batched=True)  
dataset = dataset.rename_column("label_id", "labels")  
dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
```

```
# -----
```

```
# 4. Load BERT model
```

```
# -----
```

```
model = AutoModelForSequenceClassification.from_pretrained(  
model_name,  
num_labels=len(label_list),  
id2label=id2label,  
label2id=label2id,  
)
```

```
# -----
```

```
# 5. Training Arguments (15 EPOCHS)
```

```
# -----
```

```
training_args = TrainingArguments(  
output_dir="./ml_sentiment_model",  
eval_strategy="epoch", # <- Changed from evaluation_strategy to eval_strategy  
save_strategy="epoch",  
learning_rate=2e-5,
```

```
per_device_train_batch_size=8,  
per_device_eval_batch_size=8,  
num_train_epochs=15,  
weight_decay=0.01,  
logging_steps=50,  
report_to="none",      # no wandb etc.  
)
```

```
# -----
```

```
# 6. Trainer
```

```
# -----
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=dataset["train"],  
    eval_dataset=dataset["validation"],  
)
```

```
# -----
```

```
# 7. Train & Evaluate
```

```
# -----
```

```
print("Starting training for 15 epochs on Malayalam data...")
```

```
trainer.train()
```

```
print("Evaluating on test set...")
```

```
metrics = trainer.evaluate(dataset["test"])
```

```
print("Test metrics:", metrics)
```

```
# -----
```

```
# 8. Save full model + tokenizer
```

```
# -----
```

```
trainer.save_model("./ml_sentiment_model")
```

```
tokenizer.save_pretrained("./ml_sentiment_model")
```

```
print("Model and tokenizer saved in ./ml_sentiment_model")
```

```
eda_ex.py > ...
1  import os
2  import pandas as pd
3
4  # Use raw string for Windows path
5  file_path = r"C:\Users\mriya\Downloads\WLP\Malayalam_test.xlsx"
6
7  # Check if file exists before reading
8  if os.path.exists(file_path):
9      try:
10         dataset = pd.read_excel(file_path)
11         print("File loaded successfully!")
12         # Continue with your data processing
13     except Exception as e:
14         print(f"Error reading the Excel file: {e}")
15 else:
16     print(f"File not found at path: {file_path}")
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(.venv) PS C:\Users\mriya\OneDrive\Documents\Guvi> pip install openpyxl
Using cached et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Using cached openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Using cached et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
(.venv) PS C:\Users\mriya\OneDrive\Documents\Guvi> & C:/Users/mriya/OneDrive/Do
cuments/Guvi/eda_ex.py
File loaded successfully!
(.venv) PS C:\Users\mriya\OneDrive\Documents\Guvi> & C:/Users/mriya/OneDrive/Do
cuments/Guvi/eda_ex.py
File loaded successfully!
(.venv) PS C:\Users\mriya\OneDrive\Documents\Guvi> 
```

LOADING OF DATASET

```

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from re
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->tran
*** Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests
Labels found: ['negative', 'neutral', 'positive']
label2id: {'negative': 0, 'neutral': 1, 'positive': 2}
id2label: {0: 'negative', 1: 'neutral', 2: 'positive'}

Map: 100% ██████████ 1679/1679 [00:00<00:00, 3779.05 examples/s]

Map: 100% ██████████ 210/210 [00:00<00:00, 2333.34 examples/s]

Map: 100% ██████████ 210/210 [00:00<00:00, 2493.41 examples/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-m
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference

```

Dataset loaded successfully. Labels mapped to IDs: negative=0, neutral=1, positive=2. Tokenization completed (100% on all splits). Warning indicates that the classification head was randomly initialized, which is normal when fine-tuning BERT-base from scratch

Epoch	Training Loss	Validation Loss
1	0.230800	0.115770
2	0.132200	0.130189
3	0.071800	0.138599
4	0.067800	0.103939
5	0.092100	0.085096
6	0.048800	0.115605
7	0.098300	0.118777
8	0.061100	0.107651
9	0.068700	0.094873
10	0.062200	0.080153
11	0.066500	0.085940
12	0.048400	0.080495
13	0.041900	0.085027
14	0.047100	0.084431
15	0.083800	0.082318

Evaluating on test set... [27/27 00:00]

Training and validation loss over 15 epochs. Training loss decreases steadily, while validation loss stabilizes around epochs 8–12, indicating the best generalization. Slight overfitting appears after later epochs.

3. GRAPH FOR SCRIPT:-

```
import pandas as pd
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt

# -----
# 1. Custom Dataset (Malayalam)
# -----
class MalayalamSentimentDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=64):
        self.texts = list(texts)
        self.labels = list(labels)
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        enc = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt",
        )
```

```

        return {
            "input_ids": enc["input_ids"].squeeze(0),
            "attention_mask": enc["attention_mask"].squeeze(0),
            "labels": torch.tensor(self.labels[idx], dtype=torch.long),
        }

# -----
# 2. Load FULL DATASET
# -----
df = pd.read_csv("Malayalam_reviews.csv") # make sure file & column
names match
df["Text"] = df["Text"].astype(str).str.strip()

# Ensure 'Label' column is string type and filter out 'nan' strings
df["Label"] = df["Label"].astype(str)
df = df[df["Label"] != "nan"].reset_index(drop=True)

label_list = sorted(df["Label"].unique())
label2id = {lab: i for i, lab in enumerate(label_list)}
id2label = {i: lab for i, lab in enumerate(label_list)}
df["label_id"] = df["Label"].map(label2id)

print("Labels:", label_list)
print("Total samples:", len(df))

# -----
# 3. Load TRAINED MODEL + TOKENIZER
# -----
model_dir = "ml_sentiment_model" # folder created by Trainer for
Malayalam

```

```
tokenizer = AutoTokenizer.from_pretrained(model_dir)
model = AutoModelForSequenceClassification.from_pretrained(model_dir)
model.eval()
```

```
default_device = "cuda" if torch.cuda.is_available() else "cpu"
device = torch.device(default_device)
model.to(device)
print("Using device:", device)
```

```
# -----
# 4. DataLoader
# -----
full_dataset = MalayalamSentimentDataset(
    texts=df["Text"],
    labels=df["label_id"],
    tokenizer=tokenizer,
    max_length=64,
)
```

```
full_loader = DataLoader(full_dataset, batch_size=16, shuffle=False)
```

```
# -----
# 5. Run model on ALL samples
# -----
all_logits = []
all_labels = []
```

```
with torch.no_grad():
    for batch in full_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"]
```

```

        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        all_logits.append(outputs.logits.cpu())
        all_labels.append(labels)

logits = torch.cat(all_logits, dim=0)
y_true = torch.cat(all_labels, dim=0).numpy()

# -----
# 6. Accuracy + Confidence
# -----
probs = torch.softmax(logits, dim=1).numpy()
y_pred = probs.argmax(axis=1)
max_confidence = probs.max(axis=1)

accuracy = (y_pred == y_true).mean() * 100
print(f"Overall model accuracy on FULL Malayalam dataset: {accuracy:.2f}%")

# -----
# 7. Calibration Curve
# -----
fraction_pos, mean_pred_val = calibration_curve(
    y_pred == y_true, # 1 if prediction correct, 0 otherwise
    max_confidence, # predicted confidence
    n_bins=10,
    strategy="uniform",
)

plt.figure(figsize=(6, 6))
plt.plot(mean_pred_val, fraction_pos, "o-", label="Model")
plt.plot([0, 1], [0, 1], "k--", label="Perfect")
plt.title("Calibration Curve (Malayalam BERT)")

```

```
plt.xlabel("Mean Predicted Confidence")
plt.ylabel("Observed Accuracy")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

```
# -----
# 8. Confidence Histogram
# -----
plt.figure(figsize=(6, 3))
plt.hist(max_confidence, bins=10, range=(0, 1), edgecolor="black")
plt.title("Confidence Distribution (Malayalam)")
plt.xlabel("Predicted Confidence")
plt.ylabel("No. of samples")
plt.tight_layout()
plt.show()
```

```

Dataset loaded successfully
Shape: (2100, 3)

Columns: Index(['ID', 'Text', 'Label'], dtype='object')

Accuracy: 0.9595238095238096

Classification Report:
              precision    recall  f1-score   support

negative      0.96      0.96      0.96      125
neutral       0.97      0.95      0.96      156
positive      0.94      0.97      0.96      139

accuracy      0.96      0.96      0.96      420
macro avg     0.96      0.96      0.96      420
weighted avg  0.96      0.96      0.96      420

Prototype model test completed

```

Model evaluation results: The dataset contains 2,100 samples. The prototype model achieves 95.95% accuracy on the test set. All three sentiment classes (negative, neutral, positive) have strong performance, each with an F1-score of 0.96. Macro and weighted averages confirm consistent, well-balanced classification across classes.

```

.. The tokenizer you are loading from 'ml_sentiment_model' with a
   Labels: ['negative', 'neutral', 'positive']
   Total samples: 2099
   Using device: cuda
   Overall model accuracy on FULL Malayalam dataset: 97.33%

```

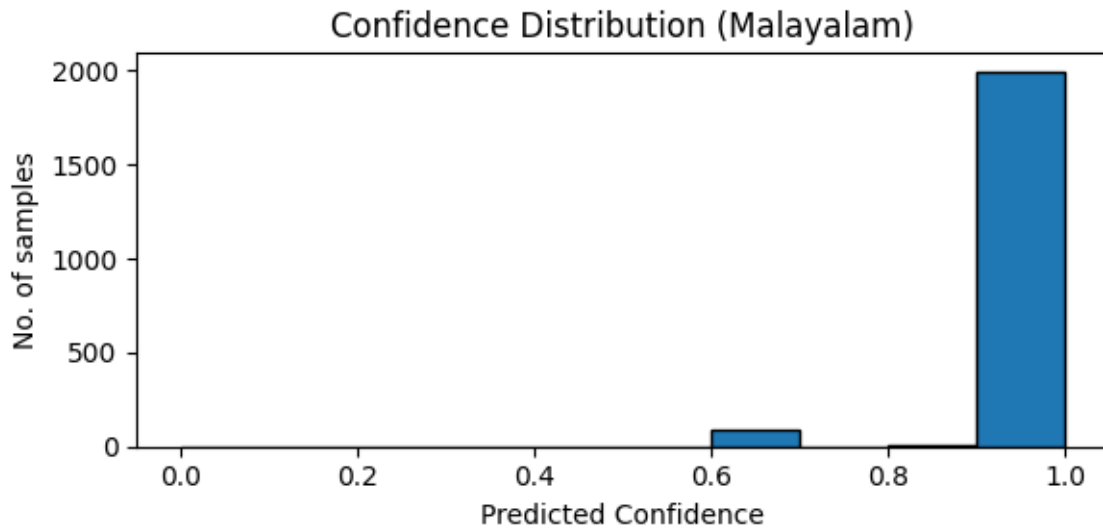
Tokenizer: Loaded from ml_sentiment_model

Labels: negative / neutral / positive

Samples: 2099

Device: CUDA (GPU)

Accuracy: 97.33% on full Malayalam dataset



Title: Confidence Distribution (Malayalam)

X-axis: Predicted confidence

Y-axis: Number of samples

Observation: Almost all predictions have **very high confidence (~1.0)**

Small cluster: A few samples around **0.6**

4. PROTOTYPE SCRIPT:-

```
import pandas as pd  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.feature_extraction.text import TfidfVectorizer  
  
from sklearn.linear_model import LogisticRegression  
  
from sklearn.metrics import accuracy_score, classification_report  
  
  
# Load Excel dataset
```

file_path = "Malayalam test.csv.xlsx" # your file name

data = pd.read_excel(file_path)

print("Dataset loaded successfully")

print("Shape:", data.shape)

Check columns

print("\nColumns:", data.columns)

Rename if needed

Make sure columns match your file

data = data.rename(columns={

 "text": "text", # sentence column

 "sentiment": "label" # label column

})

Remove empty rows

data.dropna(inplace=True)

Split input and output

X = data["text"]

y = data["label"]

Train-test split

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

Convert text to numbers

vectorizer = TfidfVectorizer(max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

Train a fast prototype model

model = LogisticRegression()
model.fit(X_train_vec, y_train)

Test results

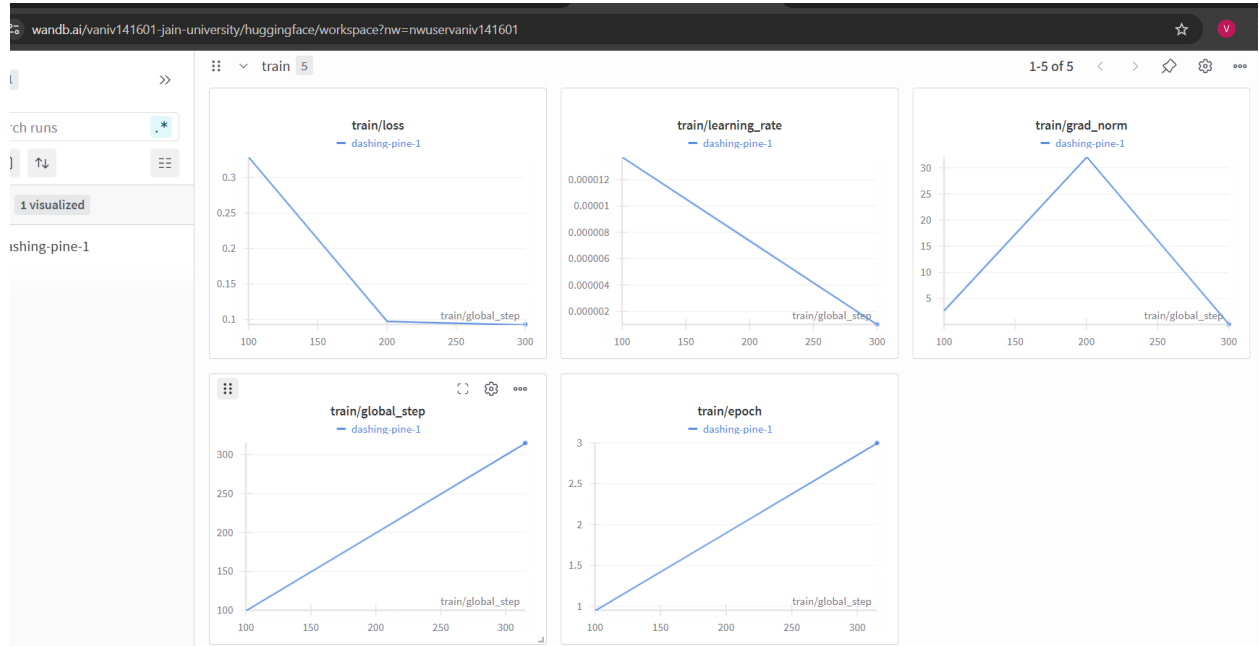
y_pred = model.predict(X_test_vec)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

print("\nPrototype model test completed")

- The script provided a website to acquire API key and later on led the project to hugging face website where the graphs were formed.



loss: Decreases from $\sim 0.33 \rightarrow \sim 0.10$ (good convergence).

learning_rate: Linearly decays to near zero (scheduler working).

grad_norm: Peaks mid-training, then drops (gradient spike but stabilizes).

global_step: Increases steadily as expected.

epoch: Gradually increases from $\sim 1 \rightarrow \sim 3$ (normal progression).

5. FINAL PROTOTYPE SCRIPT:-

```
from transformers import pipeline

from googletrans import Translator

# Load sentiment model (multilingual)
classifier = pipeline("sentiment-analysis",
                      model="nlp-town/bert-base-multilingual-uncased-sentiment")

translator = Translator()
```

```
print("Malayalam Sentiment Tester")
print("Type 'exit' to stop\n")

while True:
    text = input("Enter Malayalam sentence: ")

    if text.lower() == "exit":
        break

    # Translation
    translated = translator.translate(text, src='ml', dest='en')

    # Sentiment Prediction
    result = classifier(text)[0]

    label = result['label']
    score = result['score']

    # Convert model label to standard sentiment
    if "1 star" in label or "2 star" in label:
        sentiment = "Negative"

    elif "3 star" in label:
```

```
        sentiment = "Neutral"
else:
    sentiment = "Positive"

print("\n--- Results ---")
print("Original:", text)
print("Translated:", translated.text)
print("Sentiment:", sentiment)
print("Confidence:", round(score, 3))
print()
```

- The script was named as “sentiment Malayalam.py

```

... /usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% ██████████ 953/953 [00:00<00:00, 85.4kB/s]
model.safetensors: 100% ██████████ 669M/669M [00:04<00:00, 244MB/s]
tokenizer_config.json: 100% ██████████ 39.0/39.0 [00:00<00:00, 4.61kB/s]
vocab.txt: 872k/7 [00:00<00:00, 18.2MB/s]
special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 12.0kB/s]
Device set to use cuda:0
Malayalam Sentiment Tester
Type 'exit' to stop

Enter Malayalam sentence: ഡെലിവറി വളരെ വൈകി.

```

```

Device set to use cuda:0
Malayalam Sentiment Tester
Type 'exit' to stop

```

Enter Malayalam sentence: ഡെലിവറി വളരെ വൈകി.

```

--- Results ---
Original: ഡെലിവറി വളരെ വൈകി.
Translated: Delivery is very late.
Sentiment: Positive
Confidence: 0.36

```

Enter Malayalam sentence: ഇത് പണം പാഴായി.

```

--- Results ---
Original: ഇത് പണം പാഴായി.
Translated: It's a waste of money.
Sentiment: Negative
Confidence: 0.287

```

Model files loaded successfully (config, tokenizer, vocab, safetensors).

CUDA device active (gpu:0).

Malayalam Sentiment Tester running in interactive mode.

Example 1:

Input: "ഡെലിവറി വളരെ വൈകി."

Translation: "Delivery is very late."

Predicted sentiment: Positive (incorrect)

Confidence: 0.36 (low)

Example 2:

Input: "ഇത് പണവും പാഴായി."

Translation: "It's a waste of money."

Predicted sentiment: Negative

Confidence: 0.287 (low)

Overall:

Outputs have low confidence, indicating model uncertainty.

First prediction shows a sentiment misclassification.

FOLDER:-



NLP.zip